RnD Project Report: Mimicking Networks for $K_{3,3}$-free and K_5-free Graphs

Gargi Bakshi Sathwika Reddy
Guide: Prof. Rohit Gurjar

March 3, 2026

Contents

1 Introduction **1**
1.1 $K_{3,3}$-free Graphs . . . 2
1.2 K_5-free Graphs . . . 2

2 Definitions **2**
2.1 Feasible Flows Definition . . . 2
2.2 Min-Cut Definition . . . 2
2.3 Proof of Equivalence . . . 2
2.3.1 Definition 1 ⇒ Definition 2 . . . 2
2.3.2 Definition 2 ⇒ Definition 1 . . . 3

3 Approach **3**

4 Two Sum of Mimicking Networks of Graphs **4**
4.1 Case 1: Both Terminals . . . 4
4.2 Case 2: One or Both Non-Terminals . . . 5

5 Mimicking Network Construction Algorithm **5**
5.1 Final Algorithm . . . 5
5.2 Optimizations to Bound the Number of Terminals . . . 6
5.2.1 Trailing Trees With Only Artificial Terminals . . . 6
5.2.2 Chain of Components Without True Terminals Between Two Components Having Terminals . . . 7

6 Future Work **8**

1 Introduction

A **mimicking network** is a more compact version of a network represented by an undirected graph with some vertices called **terminals** which are preserved in the mimicking network. Hagerup et. al. [1] defined this concept and proved that such a network of size at most 2^{2^k} always exists, where k is the number of terminals. Krauthgamer et. al. [2] gave a method for generating mimicking networks for planar k-terminal networks that are of the size $\mathcal{O}(k^2 2^{2k})$.

Wagner's theorem characterizes the planar graphs as exactly those graphs that do not have as a minor either a complete graph K_5 on five vertices or a complete bipartite graph $K_{3,3}$ with three vertices on each side of its bipartition. Therefore, the set of planar graphs is the intersection of the set of $K_{3,3}$-minor-free and K_5-minor-free graphs and several results that are true for planar graphs can be extended to $K_{3,3}$-minor-free and K_5-minor-free graphs as well. Our RnD project is formulating a method to generate mimicking networks for $K_{3,3}$-minor-free and K_5-minor-free graphs with k-terminals that have a similar spatial bound.

1.1 $K_{3,3}$-free Graphs

The $K_{3,3}$ graph is a complete bipartite graph with six vertices divided into two subsets of three vertices each and an edge between every vertex in one subset and every vertex in the other subset. It is a non-planar graph. A $K_{3,3}$-free graph is one which does not have $K_{3,3}$ as a minor.

1.2 K_5-free Graphs

The K_5 graph is a complete graph with five vertices, where each pair of vertices is connected by a unique edge. It is a non-planar graph. A K_5 graph is one which does not have K_5 as a minor.

Graph Minor

An undirected graph H is called a minor of the graph G if H can be formed from G by deleting edges, vertices and by contracting edges. An edge contraction is an operation that removes an edge from a graph while simultaneously merging the two vertices it used to connect.

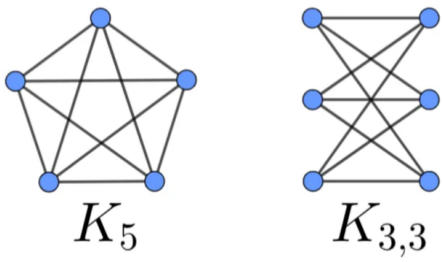

Figure 1: $K_{3,3}$ and K_5 graphs

2 Definitions

We have two definitions for the mimicking network of a graph: the Feasible Flows definition and the Min-Cut definition [2]. The starting step of our project was understanding the two definitions and establishing their equivalence. Subsections 2.1, 2.2, and 2.3 detail the definitions and the proof of equivalence.

2.1 Feasible Flows Definition

For a network G, another network H with the same number of terminals is called a mimicking network if the set of feasible flows of G is the same as the set of feasible flows of H.

A flow is given by a flow vector, an n-dimensional vector where the ith entry represents the flow in or out of the ith terminal for a graph with n terminals. A flow is feasible if it satisfies the capacity constraints and flow conservation constraints in the network.

2.2 Min-Cut Definition

For a network G, another network H with the same number of terminals is called a mimicking network if the min-cut of G is the same as the min-cut of H for every bipartition of the terminals.

2.3 Proof of Equivalence

2.3.1 Definition 1 $\Rightarrow$ Definition 2

Given two networks G and H with the same terminals such that the sets of feasible flows are the same for both networks, we have to prove that for every bipartition of terminals, the min-cut in the two networks is the same.

For any bipartition, there would either exist some flows with sources and sinks corresponding to that bipartition, or there would be no such flow.

Case 1: There exist some flows for the given bipartition
In this case, we have a maximum flow for this bipartition which is the max-flow for both graphs. To apply the max-flow min-cut theorem, we can construct equivalent networks with a single source and a single sink, with the dummy source node having edges of infinite capacity to all the source terminals and the dummy sink having edges to all the sink terminals.

Now, applying the theorem, we get that the min-cut in the two networks is equal. Note that the bipartition corresponding to this min-cut will be the same as our original bipartition along with the dummy source on the side of the source terminals and the dummy sink on the side of the sink terminals since any deviation from this will cause an edge of infinite capacity being a part of the cut, which cannot be.

Case 2: No flow exists for the given bipartition
This is only possible if the source terminals and sink terminals specified by this bipartition are disconnected components in both networks. In this case, the min-cut is zero in both networks.

With this, we prove the required implication.

2.3.2 Definition 2 $\Rightarrow$ Definition 1

Given two networks G and H with the same terminals such that for every bipartition of the terminals, the min-cut in the two networks is the same, we have to prove that the set of feasible flows is the same for both networks.

Assuming the contrary, consider a bipartition with one partition containing n sources $\{p_1, p_2, \ldots, p_n\}$ and the other containing m sinks $\{q_1, q_2, \ldots, q_m\}$ for which the flow is feasible in G but not in H.

Consider the flow, which is feasible in G but not in H. Let the ith source p_i contribute a_i flow and let the jth sink q_j absorb b_j of the flow. Then the flow vector is $[a_1, a_2, \ldots, a_n, -b_1, -b_2, \ldots, -b_m]$ with the constraint that

$$\sum_{i=1}^{n} a_i = \sum_{j=1}^{m} b_j \tag{1}$$

The min-cut in G, which equals that in H, must at least be $\sum_{i=1}^{n} a_i$.

We can construct a single-source network H' that has the same structure as H but a new terminal source node $s_{H'}$ with outgoing edges to each of the n sources in H with capacity a_i for the ith source, and a new terminal sink node $t_{H'}$ with incoming edges from each of the m sinks in H with capacity b_j for the jth sink. Then, for this bipartition, H''s behaviour is the same as that of H and therefore the min-cuts of the two networks should be equal.

Similarly, we construct G' which is equivalent to G.

We want to prove that the $s-t$ min-cuts of G' and H' are equal. Consider a set of terminals $T = \{t_1, t_2, \ldots, t_k\}$ and a bipartition in G' with $\{s_{G'}\} \cup T$ in one partition, and the corresponding bipartition in H' with $\{s_{H'}\} \cup T$ in one partition. We consider the $s-t$ cut for this bipartition for both graphs. This cut has 3 components:

1. Edges from $s_{G'}$ ($s_{H'}$) to the source terminals not in T
2. Edges present in the original graph G (H) across this cut
3. Edges from the sink terminals in T to $t_{G'}$ ($t_{H'}$)

By the construction of G' and H', components 1 and 3 are equal in the two graphs. As per definition 2 for mimicking networks, the min-cuts of G and H are equal. Since component 2 is essentially a bipartition (with one partition comprising of T) in the original graphs G ad H, the minimum value of component 2 is the min-cut for that bipartition, which is the same for both graphs. This shows that the min-cut in H' is the same as the min-cut in G', which is at least $\sum_{i=1}^{n} a_i$.

Therefore, the maxflow is at least $\sum_{i=1}^{n} a_i$ which shows that the flow with the vector $[a_1, a_2, \ldots, a_n, -b_1, -b_2, \ldots, -b_m]$ is feasible in H', which implies that it is feasible H, which leads to a contradiction. Therefore, our assumption that there exists a flow which is feasible in G but not in H is wrong.

This proves the required implication.

3 Approach

Every $K_{3,3}$-free graph can be constructed as the **two-sum** of planar components and K_5. Given two graphs G_1 and G_2, the two-sum operation of G_1 and G_2 is a graph G formed from the disjoint union

by identifying a pair of vertices $\{u, v\}$ each in G_1 and G_2 such that the resulting graph G has vertices $V(G_1) \cup V(G_2)$ and edges set $E \supseteq E(G_1) \oplus E(G_2)$ with a possible edge between u and v.

Therefore, we construct the **biconnected component tree** for a $K_{3,3}$-free graph where the leaf nodes are planar components or K_5. We then compute the mimicking networks for the planar graphs using the previously known approach and obtain constant-sized networks for K_5. Following this, we combine the mimicking networks in a way that preserves their mimicking property while retaining the required spatial bound.

In the next section, we explain how to combine the mimicking networks of two graphs using the two-sum operation.

4 Two Sum of Mimicking Networks of Graphs

Given a graph G which can be obtained using the two-sum of two graphs, G_1 and G_2 (ie, $G = G_1 \oplus G_2$), we want to obtain the mimicking network of G using the mimicking networks of G_1 and G_2.

We have two cases:

1. The two vertices of G_1 and G_2 participating in the two sum are both terminals.
2. At most one of the two vertices is a terminal.

4.1 Case 1: Both Terminals

Let the two vertices be t_1 and t_2 and let the mimicking networks of G_1 and G_2 be M_1 and M_2. If the edge between t_1 and t_2 has capacity c in G, c_1 in M_1 and c_2 in M_2, the mimicking network of G will be the two sum of M_1 and M_2 with an edge of capacity $c_1 + c_2 - c$.

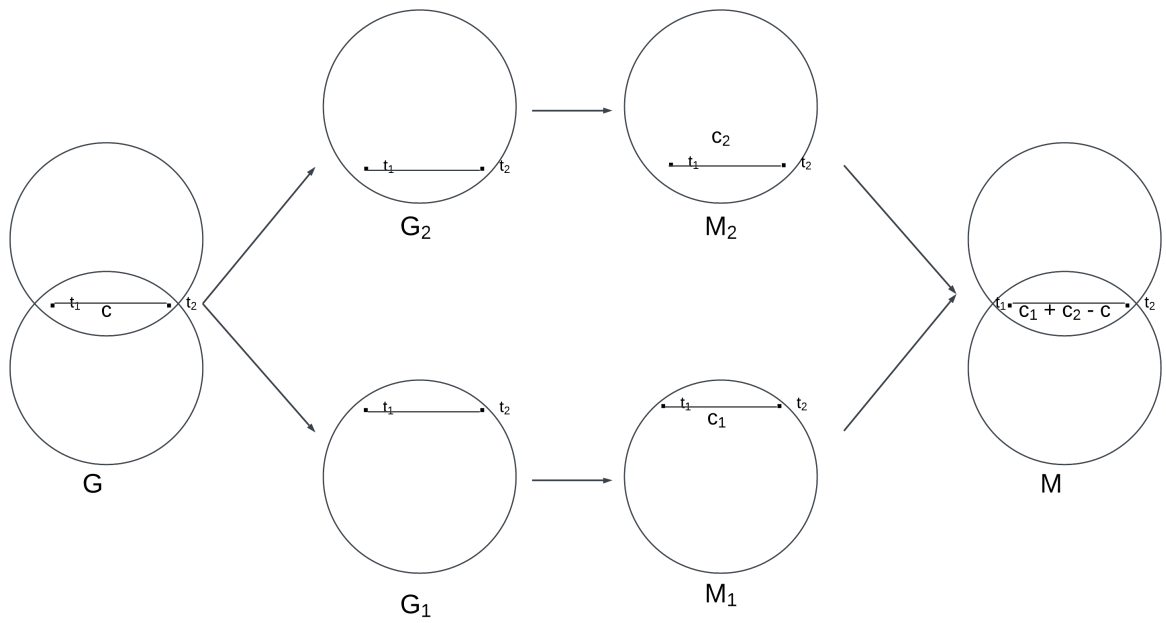

Figure 2: Obtaining the edge capacity in the mimicking network

Proof: In G, let X be the total capacity of all incoming edges from the vertices in G_1 to all vertices in the partition $\{t_1\}$ except the edge from t_2 to t_1 (capacity c) and let Y be the total capacity of all incoming edges from the vertices in G_2 to all vertices in $\{t_1\}$ except the edge from t_2 to t_1. Then the min-cut in which one partition is $\{t_1\}$ is $X + Y + c$.

The corresponding min-cut in G_1 (H_1) has value $X + c$ and in G_2 (H_2) has value $Y + c$. Since the edge between t_1 and t_2 has value c_1 (c_2) in M_1 (M_2), the total capacity of all other edges incoming to t_1 is $X + c - c_1$ ($Y + c - c_2$).

Let the mimicking network of G be M. The min-cut in G and H in which one partition is t_1 will have capacity $X + Y + c$. In H, let the edge between t_1 and t_2 have capacity c'. The min-cut in H has value $c' + (X + c - c_1) + (Y + c - c_2)$.

$$c' + (X + c - c_1) + (Y + c - c_2) = X + Y + c \tag{2}$$

$$c' = c_1 + c_2 - c \tag{3}$$

Special Cases

We can see that this formula works for the case where there is no edge between t_1 and t_2 in both M_1 and M_2. This case can only occur when there is no edge between those vertices in G as well. Therefore, c_1, c_2 and c are all 0 in this case. Considering a cut in G and M in which t_1 and t_2 are on opposite sides, we note that the max flow, in this case, would be the sum of min-cuts across the respective terminals in G_1 and G_2, which is the same as M_1 and M_2. Therefore, while combining, we would have no contribution from any $t_1 - t_2$ edge, leading to a capacity of 0.

It also holds when the capacity of the edge between t_1 and t_2 is the same in both M_1 and M_2. In this case, the min-cut in some bipartition separating t_1 and t_2 in G will be $mincut(G_1) + mincut(G_2) - c$ (assuming the edge capacity is c in G_1 and G_2 as well) since each of the two components will count that edge. Therefore, while combining M_1 and M_2, the edge should be of capacity $2c_1 - c$ to consider the effect from the components and subtract the original capacity.

4.2 Case 2: One or Both Non-Terminals

When at least one of t_1 and t_2 as defined above are non-terminals, we convert them to "artificial" terminals and then construct the mimicking networks for the sub-graphs by considering them as terminals. We convert the artificial terminals back to non-terminals after combining the two mimicking networks M_1 and M_2 using the method described above.

Introducing artificial terminals while constructing the mimicking network preserves the mimicking property. Let the original k-terminal network be G. If t is taken as an artificial terminal, then we know that in the graphs G' and its mimicking network M' with $k+1$ terminals, the min-cut across any bipartition is the same. Furthermore, every bipartition of terminals in G corresponds to a bipartition in G'. More specifically, for a bipartition of terminals in G, $(S, S^{\complement})$, we have two possible bipartitions in G' - $(S \cup \{t\}, S^{\complement})$ and $(S, S^{\complement} \cup \{t\})$ - out of which the one which maximizes the flow is the exact same as the cut in G. Therefore, M' correspondingly contains the bipartition that preserves the min-cut value as well and converting t to a non-terminal after constructing M' gives us a valid mimicking network for G.

5 Mimicking Network Construction Algorithm

Given the biconnected component tree of a $K_{3,3}$-free or K_5-free graph, we construct its mimicking network bottom-up. We describe how to construct the mimicking networks of individual components and combine them, identifying cases where we can avoid creating artificial terminals, and trying to restrict the number of terminals to $\mathcal{O}(k)$.

If we introduce artificial terminals for every component using the mechanisms outlined earlier, the total number of terminals (both artificial and real) can potentially become $\mathcal{O}(n)$, where n is the number of vertices in the original network. Due to this, we would be unable to have an upper bound on the size of the mimicking network.

5.1 Final Algorithm

We use previously known methods for constructing the biconnected component tree of the non-planar network. We define *neighbouring components* as ones that would be merged by the two-sum operation while reconstructing the network.

The algorithm is (For $K_{3,3}$-free graphs):

1. Construct the biconnected component tree for the original network to obtain planar and K_5 components
2. Construct mimicking networks for all components after introducing artificial terminals if necessary
3. Combine the neighbouring components where both components have at least one true terminal
4. While combining neighbouring components in which one has terminals and the other has no terminals, combine using the optimizations discussed below wherever applicable
5. Continue till we obtain the complete graph's mimicking network as a single component

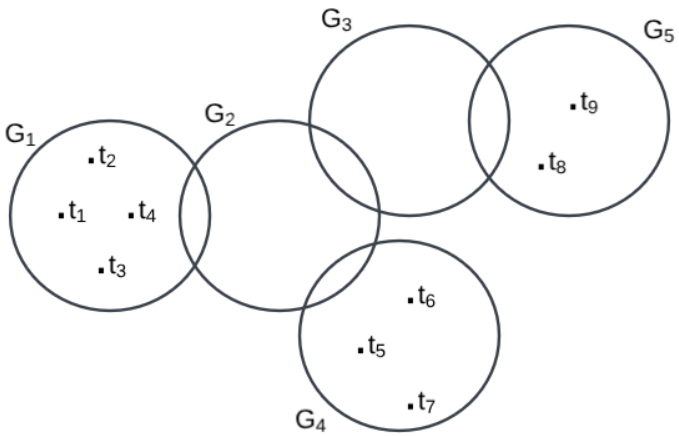

Figure 3: A case where the algorithm proposed may not bound the size

The algorithm can result in a large number of artificial terminals in some cases, which can cause the resulting mimicking network's size to blow up. For example, if we have a large number of components without any true terminals, we would end up introducing at least two artificial terminals per component. Since the number of such components can be $\mathcal{O}(n)$, we may end up with a mimicking network of size $\mathcal{O}(2^n)$. Figure 3 shows one such example. We note that in many cases in which we generate artificial terminals, the entire mimicking network for that component is not needed. The next section outlines the optimizations for such cases.

5.2 Optimizations to Bound the Number of Terminals

We discuss two cases where we can perform optimizations to reduce the number of artificial terminals, and consequently, the size of the mimicking network.

5.2.1 Trailing Trees With Only Artificial Terminals

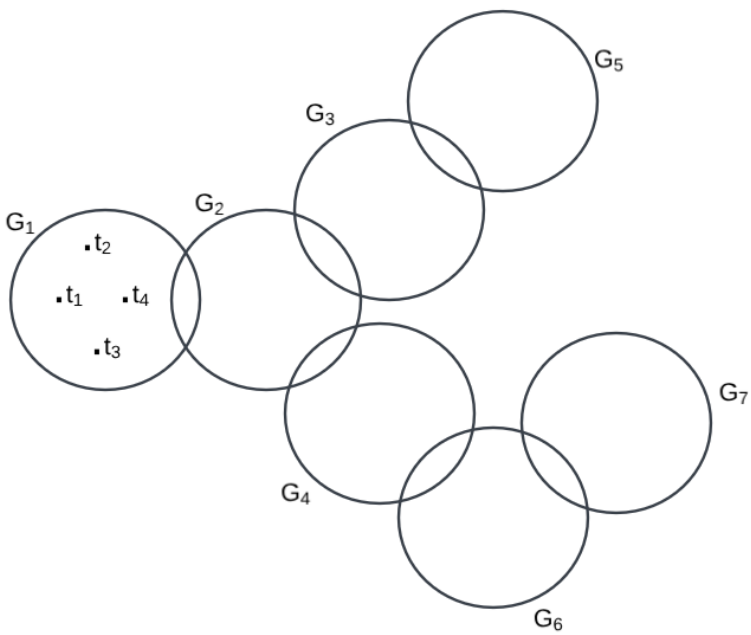

Figure 4: Trailing tree with no true terminals

While combining neighbouring components in which one has terminals and the other is connected to a set of components that form a tree without any connections to components with terminals, we first obtain

the combined mimicking network of the set of components having no true terminals. Since the artificial terminals present in this mimicking sub-network are not necessary to preserve the min-cut properties of the mimicking network, we can replace this sub-graph with a single edge. This can be done in a bottom-up fashion:

- Obtain the mimicking network for the leaf component for which the artificial terminals had an edge of capacity c between them in the original network
- Reduce it to an edge having a capacity c_1 equal to the min-cut between the two artificial terminals in that component
- Obtain the mimicking network for the neighbouring component. Let the edge between the artificial terminals be of capacity c_2 in this network
- Merge the leaf component edge with this network, to obtain an edge of capacity $c' = c_1 + c_2 - c$ (using the formula from equation 3)

We do this till we get one component. Then we continue merging neighbouring components according to standard procedure.

For example, Figure 4 shows a graph in which G_1 is the only component with terminals. We can reduce all other components to a single edge, starting from the leaf components G_5 and G_7. We apply the optimization procedure for the two chains $G_5 - G_3$ and $G_7 - G_6 - G_4$, leading to a single modified component G_2, which itself can be reduced to an edge that gets incorporated with G_1.

We obtain the effect that an entire component will have in some bipartition by replacing it with an edge since the actual distribution of nodes across that min-cut is irrelevant. Since at each stage, we preserve the maximum flow that would have passed through a component, this approach efficiently reduces the network size while retaining the mimicking network property.

5.2.2 Chain of Components Without True Terminals Between Two Components Having Terminals

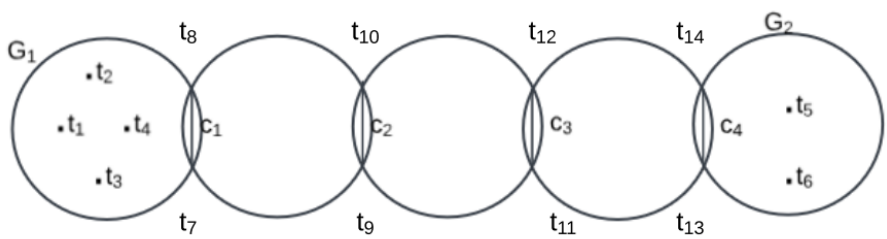

Figure 5: Chain of components with no true terminals between 2 having terminals

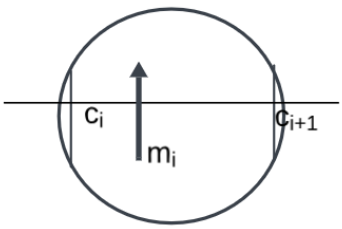

Figure 6: An individual component in a chain

If we have two components G_1 and G_2 containing true terminals, which have a chain of p components with only artificial terminals between them, then we can merge mimicking networks of G_1 and G_2 by identifying and merging corresponding artificial terminals that connect them to the chain.

The capacity for the merged edge is given by $c = c_1 \sum_{i=1}^{p}(m_i - c_i)$ where m_i is the min-cut in the ith component in the chain across the two pairs of artificial terminals, and c_i is the edge capacity between the artificial terminals common to the $(i-1)^{th}$ and i^{th} component, as shown in Figure 6.

Considering Figure 5 for reference, we note that any bipartition which separates some terminals in G_1 or G_2 (or both) necessarily separates all the artificial terminals in two sets given by $\{t_7, t_9, t_{11}, t_{13}\}$ and $\{t_8, t_{10}, t_{12}, t_{14}\}$ to maximize the min-cut. Therefore, to identify t_7 with t_{13} and t_8 with t_{14}, we need the resulting equivalent edge to have the capacity contained across that section of the min-cut. This is achieved by taking the min-cuts across the individual components. However, this causes the edges connecting artificial terminals of two intermediate terminals to be considered twice and therefore we subtract them once to obtain an edge of equivalent capacity.

6 Future Work

We need to formulate optimizations for all cases that can cause the size of the mimicking network. An example of an unhandled case, given in Figure 3, is when we have a tree of components that have only artificial terminals, such that multiple components having true terminals are attached to the ends of this tree. For the algorithm to output a mimicking network of size at most $2^{\mathcal{O}(k)}$ for every input, we need to identify all cases that can lead to a blow-up in the network's size and come up with methods to shrink the component networks.

References

[1] T. Hagerup, J. Katajainen, N. Nishimura, and P. Ragde, "Characterizing multiterminal flow networks and computing flows in networks of small treewidth," *Journal of Computer and System Sciences*, vol. 57, no. 3, pp. 366–375, 1998. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0022000098915926

[2] R. Krauthgamer and I. Rika, "Mimicking networks and succinct representations of terminal cuts," in *Proceedings of the Twenty-Fourth Annual ACM-SIAM Symposium on Discrete Algorithms*, ser. SODA '13. USA: Society for Industrial and Applied Mathematics, 2013, p. 1789–1799.